

CSC3209 - Software Architecture and Design Pattern – Practical 4

1- Given the following scenario of a project:

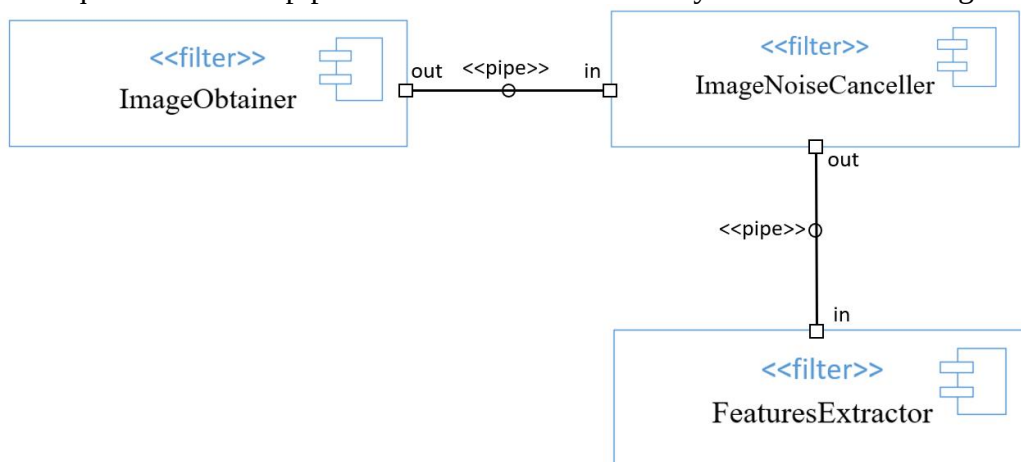
You are an architect for a facial recognition entry system at the company at which you work. You built the system having the front-end component to capture images of faces. The captured images are then passed to another component to cancel any noise of the captured images. The images with cancelled noise are then passed to another component for features extraction in which the unique features of every face image are identified. Those features are sent to a server that contains database. This database includes the details of every staff. The server is responsible to match the extracted sent features and the records in the stored database. Accordingly, if the extracted features match a record of one staff in the database, the details of that staff are displayed on a screen that is monitored by security personnel.

- Choose an appropriate architectural pattern(s)
- Justify your motivation for the selected pattern(s).
- Describe the main elements in the selected pattern(s).
- Discuss the quality attributes in your proposed system in relation to the selected pattern(s).

Solution:

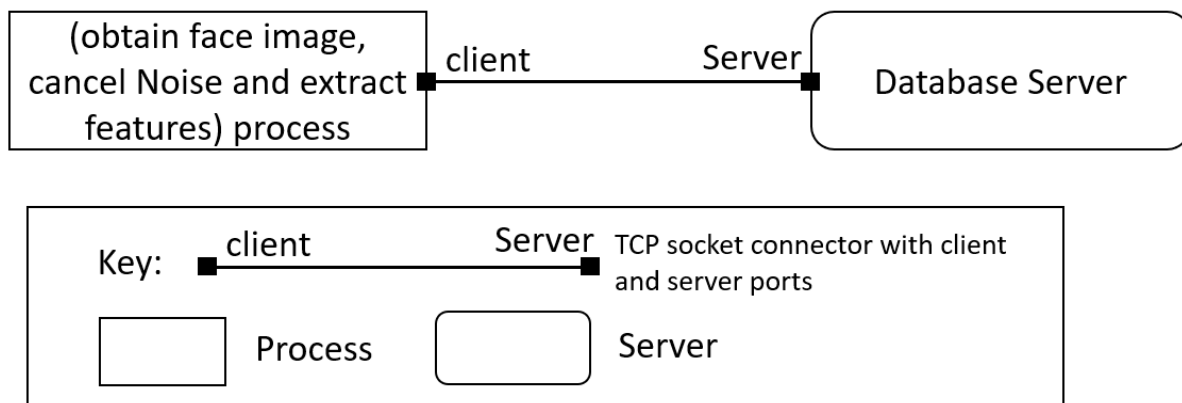
a,c) Two architectural styles are required: The pipe-and-filter and client-server.

The pipe-and-filter architectural style is to design the architecture of the software that is to extract the features of the face captured at the entrance. The style has the following filters: ImageObtainer to capture the face images, the ImageNoiseCanceller to cancel the noise in the images, and FeaturesExtractor to extract the features of faces that are required to recognize different faces. Those are connected by pipes so the output of every filter is transferred to the subsequent filter. The pipe-filter architecture for this system is shown in Figure1.



-Figure2-

Client-Server pattern is employed to use the extracted face features in obtaining the staff identity from the database. The elements of this pattern are shown in Figure2.



-Figure2-

b) The pipe and filter pattern is selected for feature extraction of faces because there is a successive transformations of streams of data. Data arrives at a filter's input port(s), is transformed and then is passed via its output port(s) through a pipe to the next filter. Also, Pipes buffer data during communication. Because of this property, filters can execute asynchronously and concurrently. This is very necessary as it is required to capture several images concurrently as several persons may enter simultaneously to the company and it is needed to recognize their faces concurrently.

Data transformation systems are typically structured as pipes and filters with each filter responsible for one part of the overall transformation of the input data. The independent processing at each step supports reuse parallelization and simplified reasoning about overall behavior.

The client-server pattern is selected because we need in the system: a client, which is responsible for face features extraction and a server, which is responsible for matching the extracted features and the records in the stored database. The client requires the service from the server which in turns provides this service to the client. In this system, only the client requests the matching service from the server.

d) Quality attributes of pipe and filter pattern

Performance: When the functionality of having several filters functioning concurrently in a way that supports parallelism, the overall system performance is improved. However, on the other hand, having many independent filters (ImageObtainer, ImageNoiseCanceller, FeaturesExtractor) can add substantial amounts of computational overhead because each filter runs as its own thread or process.

Availability: Pipe and filter systems may not be appropriate for long running computations without the addition of some form of checkpoint restore functionality as the failure of any filter (or pipe) can cause the entire pipeline to fail).

Quality attributes of client-server pattern:

Performance: Server can be a performance bottleneck

Availability: Server can be a single point of failure

Complexity: Decisions about where to locate functionality in the client or in the server are often complex and costly to change after a system has been built.

Reusability: This pattern simplifies systems by factoring out common services which are Reusable.